

MEDHAVA

Gap analysis

What is missing, what it holds up, and which gaps can be closed from inside this repository at all.

The other audit documents each measure one register. This one is the joins between them, which is where the useful findings live: a capability nobody started matters differently depending on whether four other things wait behind it.

Gap 1 — the largest one, stated plainly

94 of 113 apps are written down and not standing up. That is not a defect and it is not a surprise — a design is meant to be ahead of the build — but it is the number every other figure in this project should be read against. The design describes a business operating system; what runs today is 19 apps on the real database and 0 browser prototypes over an in-page store.

Gap 2 — nothing has ever been deployed, and it blocks the most

CAP-DEPLOY is NOT STARTED. Behind it:

- **CAP-MONITOR** — there is nothing to monitor until something runs somewhere
- every PRODUCTION-READY rung in the requirements registry, all of them empty
- the whole top of the 0–5 score: no row can reach 5
- the maturity level, which the gate caps at Prototype while this holds

And it cannot be closed from inside this environment. It needs a server, a domain and credentials, and the egress proxy here refuses everything but package registries and GitHub. This is the clearest example of a gap that is a **decision and a purchase**, not a piece of work waiting to be done.

Gap 3 — sixteen apps nothing is watching

0 apps run and carry their own self-tests, and those tests are not inside `npm test`. Nothing would notice them breaking. They score 3 — "implemented but weakly verified" — for exactly that reason, and it is the only gap in this document that can be closed without writing a feature. It is `Q01` in the build queue for the same reason.

Gap 4 — capabilities nobody has started

Capability	Why it is not started	What waits behind it
<code>CAP-DEPLOY</code> Deployment to a production environment	No server, no domain and no credentials exist.	<code>CAP-MONITOR</code> , every production claim
<code>CAP-DEVPLATFORM</code> Developer platform — public API, webhooks, SDKs	Not begun.	<code>CAP-MARKETPLACE</code> , embedded analytics
<code>CAP-STUDIO</code> Studio — building screens and flows without code	Not begun.	nothing else in the register
<code>CAP-MARKETPLACE</code> Extension marketplace	Not begun, and it cannot begin before <code>CAP-DEVPLATFORM</code> : there is nothing for a third party to extend..	nothing else in the register
<code>CAP-THREATMODEL</code> Threat model and adversarial security testing	Isolation is proven against an honest client.	nothing else in the register
<code>CAP-SCALE</code> Behaviour at production scale	Every test here runs on a handful of rows.	nothing else in the register

Gap 5 — what cannot be closed by building

`CAP-PUBLISH` **The static site published at a public URL** — Blocked on one repository setting, not on any code. The site is assembled by `mksite.js` and verified by `checksite.js`, which serves it over real HTTP and drives it in Chromium — 13 assertions, recorded. The workflow that would publish it cannot: creating a GitHub Pages site needs administration rights the Actions token does not have here, and the attempt fails with "Resource not accessible by integration". Pages has never been enabled on this repository — the runs from July failed the same way. Settings → Pages → Build and deployment → Source: GitHub Actions is the whole fix, and it is the owner's to make. `enablement: true` was expected to do it unattended and does not.

`CAP-INTEGRATIONS` **Live outside integrations** — Every marketplace, courier, tax portal, bank feed and payment provider needs live credentials, and this repository must never hold one. This cannot be raised from inside the

repository at all — it needs an environment holding secrets that no commit ever sees. Simulating one and calling it connected is the failure this whole register exists to make impossible.

This is the distinction the whole registry rests on. A BLOCKED row is not slow progress; it is progress that a commit cannot make. Reporting it as "in progress" is the failure this project has been trying to make structurally impossible.

Gap 6 — specified capabilities with nothing behind them yet

Capability	What is missing
CAP-MONITOR Monitoring, alerting and health checks	Nothing to monitor until something is deployed. Depends on CAP-DEPLOY.
CAP-AI AI gateway, agent permissions and evaluation	The provider router is the one piece that runs. There is no gateway, no permission model for what an agent may do on a company's data, and no evaluation set, so no quality claim can be made about any answer the system gives.
CAP-ANALYTICS Query engine and report builder on real data	The prototype report builder computes over an in-page store. There is no query engine against the database.
CAP-MOBILE Mobile	The shell is responsive markup that has never been opened on a phone by any check here. A responsive claim nobody has tested is a claim, and this register will not count it as a result.

Gap 7 — capability classes with no app at all

Against the 56 products the owner named, 20 have no app in this project. This is the one half of that comparison that does not depend on pages nobody could read — it is answered entirely by our own module register.

Class	Why it is a real hole, or is not
Bookings	Appointment scheduling against somebody's availability is named nowhere in the 22 modules. A trade that sells time rather than goods has nothing here.
Field Service Management	Dispatching a technician to a site, with the job, the parts and the visit, is named nowhere. A service trade has nothing here.
Lens	Remote assistance over a camera. AR / Virtual Try-On is a customer-facing fitting tool and is not the same capability.
Checkout	A hosted payment page somebody sends a customer to. Orders are named; the checkout that collects the money is not.
Payments	Payment Data Scope states which systems may ever see a card credential, which is a policy about payments and not a payments product. Taking money is named nowhere, and it is blocked besides: it needs live credentials this repository must never hold.
Vani	Contextual comments left on a document or a page. Documents are filed against the record they belong to; annotating them in place is named nowhere.
TeamInbox	A shared mailbox several people answer from. Communications sends outward; nothing here receives into a queue a team works through.
Sheet	A spreadsheet. Excel Dashboard Builder reads workbooks; it is not one.
Show	Building and presenting slides. Named nowhere in the 22 modules, and nothing here is adjacent to it — the documents this system produces are ledgers, bills and packing slips, which are printed rather than presented.
ToDo	A personal task list. Approvals and Projects & Cases are work assigned through a process, which is not the same thing as somebody's own list.
PDF Editor	Ask & Print produces PDFs and Documents & eSign files them. Editing a PDF that arrived from somewhere else is named nowhere.
Shifts	Building a rota and publishing it to staff is named nowhere, which is a real hole for a business whose floor runs in shifts — attendance is captured, and the schedule it is measured against is not.
Creator	Building an application without code. The requirements registry carries this as CAP-STUDIO, NOT STARTED: packs configure vocabulary and fields, and let nobody build a screen.
Catalyst	A developer platform to build and host services on. Carried as CAP-DEVPLATFORM, NOT STARTED — the internal API is four routes for two modules, with no versioning, no keys and no webhooks.
DataPrep	Cleaning and reshaping data before it is analysed. Master-Data Hygiene finds duplicates among customers, vendors and designs; it is not a transformation tool.
Analytics — embedded	Putting somebody else's dashboards inside your own product under your own brand. That needs the developer platform this project has not started.
Sprints	Agile iteration planning. Projects & Cases is not the same shape and does not claim to be.
BugTracker	Tracking defects in software. Quality Control inspects goods, not code.

Class	Why it is a real hole, or is not
Digital Adoption Platform	In-product walkthroughs teaching people to use the software. Named nowhere, and worth noticing for a system whose users are a shop floor.
Vertical Solutions Studio	Building an industry-specific application on the platform. Industry Packs are the nearest thing and are deliberately narrower — a pack may rename, extend and switch discretionary rules off, and may never invent a concept or carry code.

Two of these are already carried as capabilities rather than apps — a low-code builder is `CAP-STUDIO` and a developer platform is `CAP-DEVPLATFORM`, both NOT STARTED — so they appear twice on purpose, once as a competitor's product and once as our own unbuilt surface.

What this gap analysis is not

It is not a priority order. Which gap to close first is a decision that weighs what this project can verify against what the business needs next, and only the second half is the owner's to supply. `BUILD_QUEUE.md` proposes an order and states the reasoning for each position so it can be argued with.

Where every number here comes from

Nothing in this document is typed. It is generated by `node brand/delivery/website/mkaudit.js` and every figure is read at that moment from the register that owns it:

Fact	Register	Gate
modules and apps	<code>brand/site/modules.js</code>	<code>checkneutral.js</code> , <code>checkshape.js</code>
what each has reached	<code>brand/site/registry.js</code>	<code>checkregistry.js</code>
the 0–5 score and the queue	<code>brand/site/audit.js</code>	<code>checkaudit.js</code>
the capability comparison	<code>brand/site/zoho.js</code>	<code>checkzoho.js</code>
rules and their proofs	<code>brand/site/rules.js</code>	<code>checkrules.js</code>
recorded runs	<code>docs/verification/EVIDENCE.md</code>	<code>tools/evidence.js --check</code>

A figure in this document that disagrees with its register means the document is stale. Regenerate it; `npm test` refuses a stale one.

Every technical word above, in plain language

14 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

industry pack

A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue.

Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

job

One piece of work taken off the queue and done in the background.

Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

model

The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.

Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

permission

One specific thing a role is allowed to do, like approving a discount or viewing salaries.

Guchhe ki ek chaabi. Ek chaabi ek darwaza.